UPPSALA UNIVERSITET

UPTEC IT 24048

Degree project 30 credits

September 2024

Hands-on Use and Adaptation of AI in Developing and Testing Software Applications

Märta Haeggström

Master Programme in Computer and Information Engineering

UPPSALA UNIVERSITET

Hands-on Use and Adaptation of AI in Developing and Testing Software Applications

Märta Haeggström

Abstract

This thesis explores the integration of artificial intelligence (AI) in software development. Conducted at the consultancy company Amaceit AB, the study explores the current usage of artificial intelligence across various stages of the software development life cycle. It explores AI techniques such as machine learning, neural networks, deep learning, generative AI, and large language models, emphasizing their roles in enhancing efficiency and accuracy in software development. The study examines the use of AI in the planning, requirements gathering, design, development, testing, deployment, and maintenance phases of the software development process, showcasing how AI automates tasks, optimizes code generation, improves testing accuracy, and enables proactive maintenance. Furthermore, the thesis reviews popular AI tools like GitHub Copilot, Amazon CodeWhisperer, Google's Bard, Microsoft's Bing, Anthropic's Claude, and OpenAI's ChatGPT, comparing their effectiveness in code generation and analyzes their capabilities, benefits, and challenges. The study combines a literature review and a practical implementation, in the form of an AI-based web application using OpenAI's GPT-4 for automated test case generation. Findings highlight AI's effectiveness in improving coding efficiency and software quality, showing its ability to automate tasks, streamline workflows, and speed up development cycles, thereby reducing costs and enhancing software quality. While AI serves as a powerful assistant in software development by helping programmers in their everyday work lives, it is not a substitute for human expertise. The evaluation at Amaceit shows promising results in improving the efficiency of test case generation. It also addresses privacy and data security concerns related to adopting AI and proposes future steps to scale AI implementations and integrate tools directly into developer environments.

Faculty of Science and Technology

Uppsala University, Uppsala

Supervisor: Mattias Kjörk Subject reader: Ruslan Seifullaev

Examiner: Lars-Åke Nordén

Sammanfattning

Detta examensarbete utforskar integreringen av artificiell intelligens (AI) inom mjukvaruutveckling. Studien genomfördes på konsultföretaget Amaceit AB och undersöker hur AI används i olika stadier av programvaruutvecklingens livscykel idag. Den utforskar AI-tekniker som maskininlärning, neurala nätverk, djupinlärning, generativ AI och stora språkmodeller, och betonar deras roll i att förbättra effektiviteten och noggrannheten inom mjukvaruutveckling. Studien granskar användningen av AI i stadier som planering, kravinsamling, design, utveckling, testning, driftsättning och underhåll av mjukvaruutvecklingsprocessen. Den visar också hur AI kan automatisera uppgifter, optimera kodgenerering, förbättra testningens noggrannhet och möjliggöra proaktiv systemförvaltning. Arbetet undersöker populära AI-verktyg som GitHub Copilot, Amazon CodeWhisperer, Google Bard, Microsoft Bing, Anthropic Claude och OpenAI ChatGPT, jämför deras effektivitet i kodgenerering och analyserar deras förmågor, fördelar och utmaningar. Studien kombinerar en litteraturöversikt med praktisk implementering i form av en AI-baserad webbapplikation som använder OpenAI GPT-4 för automatisk generering av testfall. Resultaten visar på AI:s potential att förbättra kodningseffektiviteten och mjukvarukvaliteten genom att automatisera uppgifter, effektivisera arbetsflöden och accelerera utvecklingscykler, vilket minskar kostnader och förbättrar kvaliteten. Även om AI fungerar som en kraftfull assistent inom mjukvaruutveckling genom att stödja programmerare i deras dagliga arbete, ersätter det inte mänsklig expertis. Utvärderingen på Amaceit visar lovande resultat i att förbättra effektiviteten vid generering av testfall. Studien behandlar även integritets- och datasäkerhetsbekymmer relaterade till adoptionen av AI och föreslår framtida steg för att utöka AI-implementeringar samt integrera verktyg direkt i utvecklingsmiljöer.

Contents

1 Introduction **1**

1.1 Motivation 2

1.2 Research questions 2

1.3 Delimitations 3

1.4 Stakeholder 3

1.5 Structure of the report 4

2 Background **5**

2.1 What is Artificial Intelligence? 5

2.2 Software Development Life Cycle 6

2.3 Theoretical Framework of AI in Software Development 7

2.4 Evolution of Software Development Processes 8

3 Research methods **9**

3.1 Literature Review 9

3.1.1 Artificial Intelligence in Software Engineering 10

3.2 Practical Implementation 15

3.2.1 The choice of OpenAI API 16

3.2.2 Programming languages and tools 16

3.2.3 Creating a web application 17

3.2.4 Evaluation of the application 18

4 Results and Discussion **19**

4.1 Theoretical results 19

4.2 Practical results 26

5 Conclusions **32**

6 References **34**

A Appendix **37**

A.1 Interview questions 37

1 Introduction

Over the last few years, artificial intelligence (AI) has become a considerable part of modern life, influencing many different domains and capturing the public's attention with its many capabilities. From virtual assistants like Siri and Alexa that help manage our daily tasks to recommendation systems on platforms like Netflix and Spotify that personalize our entertainment, AI is already deeply integrated into our everyday experiences [7].

AI's impact extends further into various fields. In healthcare, it can assist in enhancing the accuracy of diagnosing medical conditions from images and anticipates health issues early on [25]. In the realm of criminal justice, AI can be employed to analyze data to predict future crime risks, potentially leading to reduced crime rates. Autonomous vehicles are another fascinating and widely spread application, where AI enables cars to navigate freely and make decisions based on real-time data [25]. These examples highlight AI's great potential across various sectors, showing its ability to handle complex tasks and simplify a human user's task.

However, as AI has advanced, concerns about its potential negative impacts on jobs and society have grown. Speculations regarding AI's ability to replace human work are many, with fears of machines gradually taking on traditionally human roles. Some envision a dystopian future where machines control a society dominated by technology [6].

Despite these fears, AI holds great potential to simplify and enhance effectiveness in software development processes when applied correctly. It's important to understand that AI won't necessarily cause human programmers to lose their jobs but instead has the potential to assist and enhance their work, making it more efficient and less labor-intensive. AI technologies could potentially offer significant opportunities for businesses, particularly in IT consulting, by reducing costs, optimizing time management, and satisfying customer demands [6]. By incorporating AI features, software development efforts can not only meet these needs but also support broader business objectives, creating extra value across different industries.

This report will investigate how AI can be leveraged to benefit human programmers in the development and testing of applications, to potentially streamline their everyday work processes.

1.1 Motivation

The integration of artificial intelligence (AI) into businesses and software development processes has become increasingly important in today's competitive landscape [6]. Modern software-intensive systems are becoming more complex, with multiple functional and non-functional requirements. Poor quality software can greatly affect the effectiveness of these systems, particularly in critical applications, potentially leading to catastrophic outcomes. Additionally, the costs of software development play a significant role in the overall expenses of these systems [4]. Understanding the significance of AI in business and its potential adoption in software development may provide opportunities for IT consulting firms to offer tailored solutions to their clients and potentially stay ahead of the curve [6]. However, the extent to which AI adoption can enable IT consulting companies to provide such solutions requires further investigation and empirical evidence.

The motivation behind this thesis comes from the recognition of AI's significant impact on business operations and the importance of IT consulting to adapt to this big change [20]. As businesses increasingly turn to AI to drive growth and innovation, the role of IT consultants becomes pivotal in facilitating successful AI adoption. However, the complexity and nuances of integrating AI into software development processes pose significant challenges for IT consulting firms.

1.2 Research questions

This project will investigate the hands-on use and adaptation of AI in developing software applications. The project will focus on the following main objectives:

- **RQ1:** In what stages of the development process is AI utilized today and in what way?
- **RQ2:** How can AI be used to streamline software development and software testing, specifically reducing time and cost?
- **RQ3:** What AI tools are commonly used in software development today and what are their benefits and challenges?

1.3 Delimitations

Given the broad scope of artificial intelligence and its numerous applications across various domains, it was crucial to narrow down the study to ensure a focused and manageable exploration. In structuring this research, several delimitations were established to maintain clarity and relevance.

The research is specifically directed at AI applications across the software development lifecycle, including planning, design, development, testing, and maintenance. This approach provides a comprehensive view of AI integration in these phases but does not extend to non-software contexts or broader business applications, thereby narrowing the scope to software development. A significant emphasis is placed on software testing for the research and the practical implementation. The practical aspect of the study involves the use of AI for generating test cases. This focus addresses the specific challenges and potentials of AI in the testing phase, providing a concrete example of its application. Consequently, the research does not cover other functionalities of AI or broader implementation scenarios.

The research was conducted within the context of Amaceit, a small IT consultancy firm. All feedback and analysis will be specific to this organization, meaning that the findings will be tailored to Amaceit's environment and might not fully represent the experiences of larger organizations or different consultancy settings. This also resulted in a very small group of participants for the user testing and analysis. While a broader perspective would have been insightful, Amaceit was the primary stakeholder for this project, and a company-specific focus was necessary to keep the research relevant and feasible.

These delimitations were carefully considered to ensure that the research remained focused and relevant, addressing specific aspects of AI in software development while providing meaningful insights within a defined context.

1.4 Stakeholder

This thesis project was conducted at Amaceit AB in Stockholm. Amaceit AB is a small consultancy company working in IT and Software Development, focusing primarily on bigger corporations and the public sector. They aim to provide sustainable solutions using modern technology advancements. The company is based in Linköping but has offices in Stockholm and Ljungby as well, with around 20 employees in total.

1.5 Structure of the report

This report is divided into five main chapters, starting with an introduction. In chapter 2, some background information about different types of AI is presented together with an overview of what software development processes look like, how AI can be applied to software development, and some history of software development processes to increase the understanding of the need of AI.

In chapter 3, the research methods for this thesis are explained and the information gathered from the literature review is presented in section 3.1.1.

Chapter 4 presents the results of the thesis and the discussion. Including the findings from the literature review and analysis results from the practical implementation. This is followed by chapter 5 where the conclusions are given.

2 Background

The following chapters give more in-depth information on all related topics to this thesis. It covers subjects such as machine learning (ML) and its relevant subtopics like neural networks and deep learning, and different types of generative AI solutions. The background also covers information about the software development life cycle (SDLC), some background on how AI can be used in this process, and some history on how AI came to be used more heavily in development.

2.1 What is Artificial Intelligence?

Artificial intelligence was defined in 1955 by Stanford Professor John McCarthy as "the science and engineering of making intelligent machines" [17]. Simply, it refers to the simulation of human intelligence processes by machines, particularly computer systems. AI includes a broad range of techniques and approaches, including machine learning, natural language processing (NLP), computer vision, and robotics, among others [13].

Machine learning is a branch of artificial intelligence focused on teaching machines to learn from data. It involves developing algorithms and models that allow computers to identify patterns within extensive amounts of unstructured data [6]. As a method in data analysis, ML enables machines to recognize patterns and make decisions or predictions based on these patterns [6]. Machine learning can be divided into different types: supervised, semi-supervised, and unsupervised learning [14]. The concept of supervision in ML refers to the data being labeled or not. In supervised learning, the dataset used to train the algorithm is labeled to classify data accurately. In unsupervised learning, unlabeled datasets are analyzed using machine learning techniques to find hidden patterns and similarities and differences in a cluster of information. Semi-supervised learning is a middle ground where smaller labeled datasets are used to guide classification from a larger unlabeled set. Semi-supervised learning is often used when it's too costly to label more data or when there is not enough labeled data for supervised learning [14].

Neural networks are a type of machine learning algorithm, based on supervised learning [14], inspired by the structure and function of the human brain [6]. They process information through layers of interconnected nodes, mimicking neurons. Each node performs a small calculation, and the network learns by adjusting the connections between these nodes to solve problems or recognize patterns in data. Furthermore, deep learning is a sub-field of neural networks where the focus lies on multiple layers (deep neural networks). Deep learning can use supervised learning without needing labeled datasets as it automatically can determine features distinguishing certain categories of

data from unstructured data in its raw form [14].

In the last year, one form of AI that has revolutionized the industry is generative AI which is a technique capable of generating new data samples resembling a given dataset, for example generating photos, music, text, or code [9]. Large language models (LLMs) are a specific subset of generative AI focused on natural language processing tasks, capable of understanding and generating human-like text [27]. They are based on neural networks and translate natural language for different tasks, for example into code. The development process with an LLM is conversational and intent-based, meaning that a human programmer has a dialogue with the LLM explaining the desired outcome.

GPT, or Generative Pre-trained Transformers, is a significant example of large language models developed by OpenAI [2]. It utilizes the transformer architecture and undergoes extensive pre-training on large text datasets to achieve exceptional language generation capabilities. GPT's importance lies in its ability to understand complex language structures, its scalability, and its versatility in performing various natural language process (NLP) tasks [2]. The relationship between these terms is explained by GPT being a specific implementation of LLMs within the broader category of generative AI, showcasing the advancements made in language generation through large-scale pre-training and transformer architectures.

2.2 Software Development Life Cycle

The software development life cycle (SDLC) is a structured approach used by software development teams to plan, design, build, test, deploy, and maintain software applications. It provides a systematic framework for managing the entire software development process from start to finish [3]. The approach outlines the steps and activities involved in each phase of software development, ensuring that projects are completed efficiently, on time, and within budget. It serves as a roadmap for developers, testers, project managers, and stakeholders, guiding them through the various stages of software development while ensuring quality, reliability, and customer satisfaction [3].

The SDLC typically consists of phases such as planning, requirements analysis, design, implementation, testing and deployment, and maintenance, as shown in Figure 1. Each phase has specific objectives, deliverables, and activities that must be completed before moving to the next phase [3]. Additionally, the SDLC can follow different models (e.g., Waterfall, Agile) depending on the project's requirements and organizational preferences. In this thesis, the SDLC serves as the foundational model for the research, although it is somewhat simplified in later discussions of the development process, with not all phases explicitly addressed.

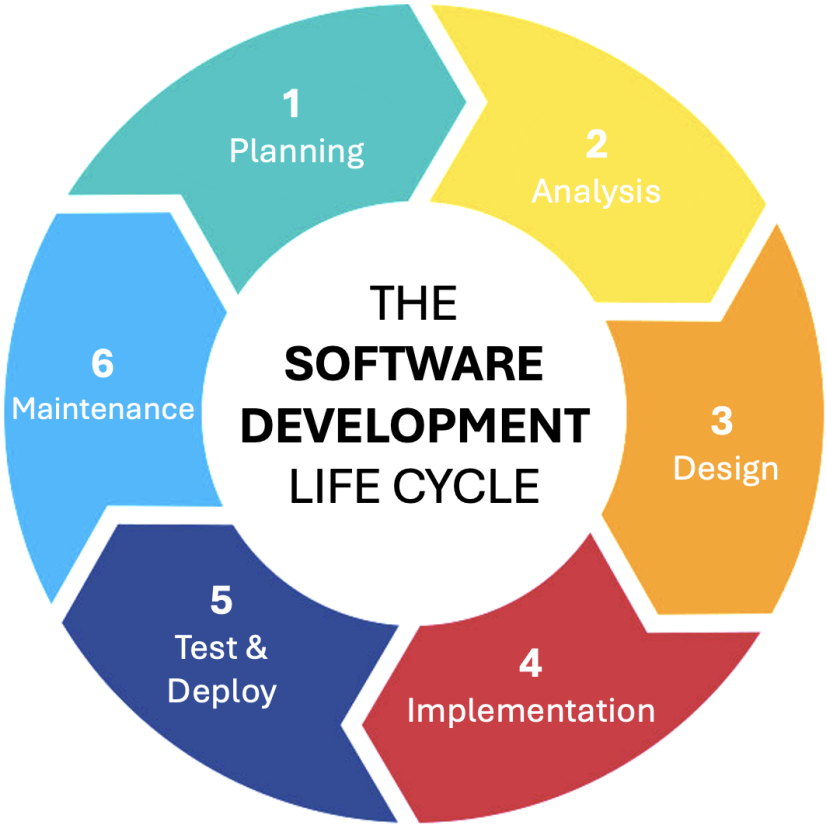

Figure 1 A circular representation outlining the six key phases of the SDLC.

2.3 Theoretical Framework of AI in Software Development

The integration of artificial intelligence into software development is reshaping the industry, unlocking vast potential to enhance efficiency, effectiveness, and overall quality. AI achieves this by automating tasks, analyzing extensive data, and identifying patterns, fundamentally transforming the software development process.

In the context of IT consulting, AI opens doors to numerous opportunities for business expansion and delivering exceptional value to customers. AI-powered solutions allow consultants to offer personalized recommendations, automate customer service interactions, and create innovative software solutions tailored to specific customer needs. By leveraging AI, IT consulting firms can distinguish themselves from competitors, elevate customer satisfaction, and drive revenue growth. Moreover, AI's versatility extends across various software development areas, covering build and deploy, release manage-

ment, continuous integration and continuous delivery, developer tools, integration/API, databases, content management system, security features, and documentation [21], [23], [22]. This integration allows for a comprehensive enhancement of the software development life cycle, streamlining processes and promoting innovation.

Despite the numerous benefits that AI can offer, its integration into software development also presents potential risks and challenges. AI algorithms and models must be carefully selected and implemented to ensure they align with the specific requirements and goals of the software project [16]. Additionally, ethical considerations and potential biases in AI algorithms must be thoroughly evaluated to avoid unintended consequences and promote responsible AI integration. With careful consideration and responsible implementation, AI can empower software developers to create more efficient, effective, and innovative software solutions, propelling the industry to new heights.

2.4 Evolution of Software Development Processes

As software development processes have developed over time, different issues and needs have emerged, making it more appropriate to incorporate AI as a tool. The transformative evolution of software development methodologies has a history from the traditional waterfall model to the iterative and agile approaches of today [18]. The waterfall model, first introduced by Winston Royce, is a sequential development process where each phase such as gathering requirements, designing, coding, testing, and launching, must be completed before the next phase begins [5]. The method showed issues such as slow feedback and poor communication and as a response, people came up with iterative and agile methods. These methods focus on giving constant feedback and assigning tasks as needed. Even though agile methods were better, they still had difficulties. This led to the development of DevOps, which combines development and operations smoothly [18].

Looking toward the future, the important role of AI-based automation in enhancing software development processes is getting more prominent, promising improvements in correctness, scalability, and timeliness. However, the misconception that better AI tools alone will yield significant advancements is important to consider, especially the importance of understanding task flows and redesigning workflows to eliminate barriers effectively.

3 Research methods

The research methods used during this project included a literature review and a practical implementation. The literature review began by exploring how various AI techniques are used in the different stages of software development and the benefits that the use of AI can give. Secondly, the review explored different AI tools for coding and how they can best be used. This study also explored which tool is best used in what context and how it can benefit the consultancy industry. The practical implementation was conducted to explore how a simple AI-based application can affect software development practices focusing on the testing phase, possibly improving time and cost efficiency.

3.1 Literature Review

The big spread of AI means that a lot of research has been done on how AI can be used in software development and in what way. Therefore, a literature review was conducted to review existing research on the topic. The review was divided into two different aspects and the articles examined in both cases were found through Google Scholar and the Uppsala University Library website using relevant keywords for the subject at hand. In addition to the keywords mentioned, prefixes and connecting words such as *application of, utilization of, using, in, for, and, or* were used. At times, the snowball technique was used to find further articles relevant to the subject and additional information [11].

The first aspect explored was what AI techniques are used in the software development process today and how they are used. For this part of the study, I wanted to find all relevant AI techniques that are used in software development to some extent and in what stage of the software engineering process they are usually utilized. To find this information, search terms alone and in combination with each other, such as *Artificial Intelligence, AI, AI-techniques, Software Engineering, Software Development, Software development process, Machine learning, Neural networks, Deep learning, Generative AI, Natural Language processing* were used.

Secondly, I wanted to find information about the use of different AI tools in programming, including code generation tools based on generative AI and LLM-based platforms. The goal of this part of the study was to explore which AI tools are most popular, which ones to use for the best results, and in what areas they perform the best. Search terms (including abbreviations of the terms) used for this review included *Artificial Intelligence, AI, Software Engineering, Software Development, OpenAI, ChatGPT, Bard, Bing, Claude, Copilot, CodeWhisperer, Generative AI, Large language models, AI-Platform, Code generation tools, Code completion tools*.

3.1.1 Artificial Intelligence in Software Engineering

For the literature review, a few main objectives were explored. First, information about how different AI techniques can be used in a software engineering process was gathered. All phases of the process were explored but the focus was mainly put on the stage of development and testing as these were of particular interest in this research. AI techniques such as generative AI, natural language processing, different forms of machine learning, and more were explored which is presented in the following section "AI in the Software Development Process".

After exploring how AI is used more generally, specific AI tools were explored to find the best ones and to identify the benefits and challenges of using them in software development. The review includes an extended comparison of five different AI tools and their use cases, including both code generation tools and regular LLM platforms. This research, based on two distinct articles is presented in the section "Comparison of Generative AI tools".

AI in the Software Development Process

Planning and Requirements gathering is the first stage of the software engineering process. In this stage, AI can be of big help to simplify tasks and make decisions based on previous projects [21]. In the planning phase, generative AI can be used to plan projects, utilizing AI's capability to use data from historical projects and analyze what went well and not, identify patterns in project planning to better estimate size and time-consumption of tasks [21].

For requirements gathering, generative AI can be used to help define the scope of projects, simplify the process of writing proposals for businesses, and even create prototypes of applications to reduce any uncertainty in wanted functionality [21]. Written text can specify the requirements through the application of NLP or by analyzing user behavior using ML [21]. Utilizing NLP, including BERT (Bidirectional Encoder Representation from Transformers), which helps computers understand word meanings in context by analyzing surrounding words, has proven beneficial in connecting separate pieces of information [15]. This makes it easier to track the origin of requirements [18].

In search-based software engineering (SBSE) work, the idea is to take problems in software engineering and turn them into optimization problems, making the goal to find the best solution out of many possible ones, according to some criteria [12]. Then, these optimization problems can be solved using computational search methods. This approach has been successful and can be used in various areas of software engineering, such as requirements gathering. In requirements gathering, SBSE techniques can be

employed to optimize software requirements based on specific metrics, ensuring better project planning and management [12].

Design as the next step in the process, can greatly utilize AI to assist in creating new system designs by providing useful insights in selecting good architecture patterns. It can also generate system designs from scratch by using the system definitions provided by human users [21]. Additionally, AI can help in areas such as analyzing weaknesses in designs and providing alternate ideas for a more uniform design. It can also help with testing of performance and scalability, documentation and to create mockups of UX designs [21].

Furthermore, SBSE techniques continue to be relevant in the design stage. They can assist in creating new system designs and optimize designs based on various metrics, ensuring a more efficient and effective design process [12].

Development is the main stage of a software engineering process, covering the coding, code review, and general creation of the actual application. As the central phase of the process, the development stage is highly adaptable and open to change, particularly when incorporating various approaches to artificial intelligence [21]. By using AI you can easily convert system design into code, get suggestions of code snippets and algorithmic improvements, check for syntax and formatting errors, and debug. This approach is expected to lessen the workload and therefore the need for a workforce in companies but also help programmers be more efficient in completing tasks [21].

The possibility of neural networks to aid in software coding has been the subject of research since the 80s, particularly the process of translating natural language into software code, a functionality that we see applied in LLMs today [6]. In general, generative AI and LLMs in particular, are big contributors to the use of AI in active development. Different programs and tools such as ChatGPT, Bard, Bing, and Github Copilot are widely used by programmers and businesses alike, helping increase the efficiency of the coding phase. These tools mentioned have different benefits, challenges, and use cases, further explored in the next section “Comparison of Generative AI tools”.

Another technique is autoencoders, a type of neural network that uses non-linear routines to process data [6]. They’re trained to understand certain elements of a dataset and to reduce redundancy or unnecessary information, which helps in tasks like reducing the size of the data or categorizing it effectively. These autoencoders consist of multiple layers, and each layer learns from the previous one to achieve a specific goal, like dimensionality reduction or feature selection [6]. They can speed up the software development process by handling tasks such as computational search, optimization, probabilistic reasoning, classification, coding, and prediction functions. Autoencoders, in particular, can simplify complex data by merging different pieces of information.

Testing and Quality assurance is one of the most important stages of the software engineering process to make sure the code works as intended and without bugs, and it's a stage with many possibilities to utilize AI. Using generative AI, tests can be generated and run on developed code to ensure requirements are being met [21]. AI capabilities can also be used in regression testing and user acceptance testing to find faults before code goes to production.

It can also aid in debugging and error handling by finding edge cases, data validation, and executing automated tests. AI's high understanding of different programming languages, frameworks, and methods, makes it able to create tests in a short time and of high quality [21].

During the testing phase, an AI technique within machine learning called multiobjective search can be applied to improve test case development and resolution [18]. Multiobjective search refers to a type of optimization problem where multiple conflicting objectives need to be optimized simultaneously, aiming to find a diverse set of solutions that represent trade-offs between the objectives [18]. The technique is effective in finding and prioritizing software crashes and suggesting fixes. Another technique used in the testing and debugging phases is hierarchical clustering which learns past fix patterns to automatically fix common bugs [18].

Another use for ML is in automated software testing where pattern recognition and machine learning techniques can be used to streamline the testing process. This involves executing test scripts repetitively to gather and interpret results [6]. Additionally, AI-powered program browsers can automatically propose code changes, further enhancing the efficiency of testing and debugging.

While not solely AI-based, fuzzing techniques, which use invalid or random data to systematically test programs, are sometimes combined with AI elements. Deep neural networks are used to identify complex code defects and improve code quality systematically [6]. In fuzzy and probabilistic work, the goal is to use artificial intelligence techniques that are good at dealing with real-world problems. These issues are usually fuzzy, meaning they're uncertain or unclear, and probabilistic, relying on probabilities [12]. This makes sense because software engineering is increasingly dealing with messy, uncertain, and incomplete information. As software becomes more integrated into our lives, it needs to handle situations that aren't always clear-cut. This applies not only to the software itself but also to the processes used to build it, many of which rely on estimates rather than exact instructions [12].

SBSE is also highly applicable in the testing and quality assurance phase. It can optimize test case development and resolution by identifying the most critical areas to test and suggesting fixes based on predefined metrics [12]. SBSE techniques ensure

thorough testing and higher software quality.

Deployment and Maintenance can heavily utilize generative AI as an assistant in various tasks. Firstly, it aids in preparing for production release by organizing software components and scheduling the deployment process effectively [21]. This involves breaking down the release into individual tasks, ensuring a systematic approach to deployment. Additionally, generative AI can monitor performance during and after deployment, offering real-time insights and statistics to track the software's behavior [21]. In the post-deployment phase, AI tools continue to provide valuable feedback and analysis, helping developers identify areas for improvement and optimize performance over time.

In the maintenance stage of software development, AI tools play an important role in finding and fixing production errors efficiently. These tools keep systems secure by deploying patches quickly, ensuring the software remains strong against potential threats [21]. Additionally, AI can analyze user behavior to identify patterns and irregularities, helping developers to address issues ahead of time. AI tools can also provide customized reports to application owners, offering valuable insights into system performance and user engagement. Moreover, AI helps ensure compliance with regulatory requirements and security standards, enhancing the overall reliability and trustworthiness of the software product [21]. Through these capabilities, AI significantly contributes to the maintenance and support efforts, ensuring optimal performance, security, and compliance throughout the software process.

Finally, in the maintenance stage, SBSE plays an important role in diagnosing and troubleshooting production defects efficiently. By optimizing maintenance processes, SBSE ensures quick deployment of patches and early action to resolve issues. This optimization contributes to the overall reliability and trustworthiness of the software product [12].

Comparison of Generative AI tools

Something that has become quite apparent is that different forms of generative AI such as large language models and GPT-based tools play a crucial role in the incorporation of AI in Software development. In this literature review, comparison work has been incorporated to evaluate the most effective use of these tools and determine the best platform to use in different cases. Two comprehensive studies have been summarized here, one comparing three popular tools used for code generation; GitHub Copilot, Amazon CodeWhisperer, and OpenAI's ChatGPT, and the second covering a comparative analysis of four LLMs; Google's Bard (now Gemini), Microsoft's Bing, Anthropic's Claude and once again OpenAI's ChatGPT. Google's platform now called Gemini, formerly known as Bard will further be referred to as *Google's Bard* or just *Bard* for simplicity,

since this was the name of the platform at the time of the study.

The first study, conducted by Yetiştiren et al. [26] showed a comparative analysis of three popular AI-driven code generation tools: GitHub Copilot, Amazon CodeWhisperer, and ChatGPT. These tools are differentiated from regular code completion tools by focusing on their ability to generate entire code blocks based on prompts or descriptions provided by the user. The evaluation criteria included various dimensions of code quality, including validity, correctness, security, reliability, and maintenance. The study revealed notable insights into the performance of each tool in these aspects, showcasing a valuable result of how these tools can best be used.

The most important aspect of this study was the showcasing of the correctness of the tools' generated code. The correctness was measured as the number of passed unit tests divided by the total number of unit tests for that specific problem. An average correctness score was then calculated by adding all correctness scores and dividing by the number of problems. Among the evaluated tools, ChatGPT got the highest success rate, managing to generate the most correct code for a given problem with a score of 65.2%. Additionally, it achieved 22.6% partially correct solutions but it was not without flaws, as 12.2% of its generated solutions were incorrect. In comparison, GitHub generated 46.3% entirely correct code and Amazon managed 31.1% correctness.

Another observation from the study was the presence of repeated instances of code smells in the generated code across all three tools. Code smells serve as indicators of underlying issues within the code base or application without necessarily being visible bugs in the code [1]. Despite the similarity in the code smells, the estimated time required to address these code smells varied among the tools. Amazon CodeWhisperer showed the shortest time requirement for fixing the code smells (5.6 minutes), followed by ChatGPT (8.9 minutes), and GitHub Copilot (9.1 minutes).

The study also explored how the user's precision in naming functions and choosing parameter names impacted the tools' ability to perform. First, the study looked at the impact of input parameters' quality and it was observed that lower-quality input parameters led to decreased percentages of correct code generation across all tools. Additionally, the use of generic or placeholder function names (e.g., 'foo') negatively impacted the outcomes, although ChatGPT displayed relatively better performance compared to Amazon in this aspect. The study emphasized the significance of accurate and clear function descriptions for optimizing the performance of AI-driven code-generation tools.

The second study conducted by Coello et al. [8] presents a comprehensive study comparing the coding effectiveness of ChatGPT, powered by GPT-3.5 and GPT-4, with other LLMs such as Google's Bard, Microsoft Bing, and Anthropic's Claude. The study high-

lights the distinct features of Generative Pre-trained Transformer (GPT) models, emphasizing their generative AI capabilities that set them apart from conventional LLMs, and evaluates their performance in comparison. Since not all models could offer API access for various reasons, the study used the web interfaces of all models. The study employed the Basic Python Programming (MPPP) dataset, consisting of approximately 1000 problems tailored for entry-level programmers. From this dataset, a subset of 460 problems was selected for analysis.

The evaluation results revealed clear differences in coding effectiveness among the tested models. The point system used in measuring the effectiveness of the code gives a score based on the outcome of the assertion tests. If an assertion test succeeds, one point is awarded, whereas if it fails, zero points are awarded. The cumulative score across all assertion tests determines the overall effectiveness of the code. When testing the tools, Claude exhibited the lowest performance, scoring only 71.43% of the maximum possible points (875 out of 1225), followed closely by Bard with 76.16% (933 points). In contrast, ChatGPT using the GPT-3.5 model scored 83.18% (1019 points), while Bing scored slightly lower at 81.96% (1004 points) even though it's based on a GPT-4 model. ChatGPT using the GPT-4 model outperformed all other models with a score of 87.51% (1072 points), highlighting its superiority in code generation tasks.

An analysis of the generated code revealed that non-GPT models tended to produce longer and unnecessarily complicated code, often getting stuck in loops. In contrast, GPT models generated shorter and more concise code while maintaining functionality. Bard exhibited the highest average number of lines of code (LOC), indicating lower-quality code, whereas Bing produced the highest-quality code in terms of LOC. To determine how well the models could comprehend feedback from a user and correct its errors, previously failed problems were retested with feedback. In this test, GPT-4 demonstrated the ability to improve and ultimately resolve all errors. In comparison, Bard acknowledged errors but failed to correct them in new versions of the code.

The study also outlined several limitations associated with LLMs, including inconsistent output, the need for effective communication of feedback, differences in programming style, and potential compatibility issues with existing codebases. This study led to the conclusion that LLMs have great potential as coding assistants but are not suitable replacements for human programmers.

3.2 Practical Implementation

To further explore how AI can be used in software development, a practical implementation was performed in the form of an AI-based web application that given a function

description creates test cases for said function. The implementation was done using OpenAI's API to utilize the GPT-4 model together with JavaScript and Node.js.

3.2.1 The choice of OpenAI API

For this implementation, the OpenAI API was chosen as the API to use. Other options that were considered included Anthropic's Claude API and Google's Gemini API. The reason for not choosing one of these alternatives was due to their lack of development compared to OpenAI, as they are still in an ongoing training phase (as of March 2024). Both Gemini and Claude show strong potential for the future, but they are still in development and not yet fully advanced, making them currently insufficient for certain applications [24]. Additionally, neither of these APIs is based on the GPT-4 model like OpenAI which is considered the most powerful model to date, promising the highest quality results. OpenAI may be somewhat more expensive at the moment, but the competition in the market are expected to lower prices rather than increase them. Therefore, it's more important to get the highest quality output than to consider the price at this stage [24].

3.2.2 Programming languages and tools

The practical implementation was conducted in JavaScript along with the Node.js framework. When utilizing the OpenAI API, the most suitable programming languages are JavaScript (with Node.js) and Python. This recommendation is based on the availability of official libraries for these languages, which facilitate seamless integration and streamline the development process. The reason for not choosing Python in this project was mostly due to the stakeholder Amaceit's existing tech stack. At Amaceit, JavaScript is predominantly used, making it the clear choice for simplifying any future work on the application. JavaScript was also chosen due to its suitability for this implementation. From the beginning, the plan was to integrate the OpenAI API into either a Visual Studio Code (VSCode) extension or a web application built with HTML. Consequently, JavaScript was preferred because of its compatibility with front-end projects and its status as the preferred language for building VSCode extensions.

Additionally, the language was chosen due to personal preference and prior experience. While I have extensive experience with Python, recent work with JavaScript in front-end projects has rendered it more familiar and accessible, particularly given the substantial front-end component of this project. By choosing a familiar language the learning curve associated with new technologies could be minimized. Lastly, the framework Node.js

was chosen because it works well with the OpenAI API when using JavaScript as mentioned earlier, making the development process smoother.

3.2.3 Creating a web application

When starting this project, how an AI API should be used was not certain. Ideas such as creating a VSCode Extension or other IDE plugin were explored and could have been of great value to create. This was unfortunately more complicated to develop than expected and would take too much time. Ultimately the decision fell on making a web application because this turned out to be the most feasible within the given time frame.

The web application is structured into two separate components that work together with the external OpenAI API: the client side and the server side, as shown in Figure 2. On the client side, an HTML page is styled with CSS and powered by a JavaScript script that manages all client-side functionality. This includes presenting the user with a form where a programming language and a function description can be entered. When the user presses the button to generate test cases, the JavaScript script captures these inputs, handles pasting events to ensure correct formatting in the text boxes, and sends POST requests to the server with the function description.

On the server side, Node.js is utilized to create a simple server that hosts the application and communicates with the external OpenAI API. The server processes the inputs from the client side and sends them to the API to retrieve appropriate test cases. The server sends the input as a user prompt, along with a predefined system prompt, to the OpenAI API. The system prompt guides the API to generate appropriate test cases based on the provided function description. Once the server receives the test cases from the API, it sends them back to the client side, where the JS script then displays the generated test cases to the user. This seamless interaction between the client and server sides, along with the integration of the OpenAI API, ensures efficient processing and accurate results for the user.

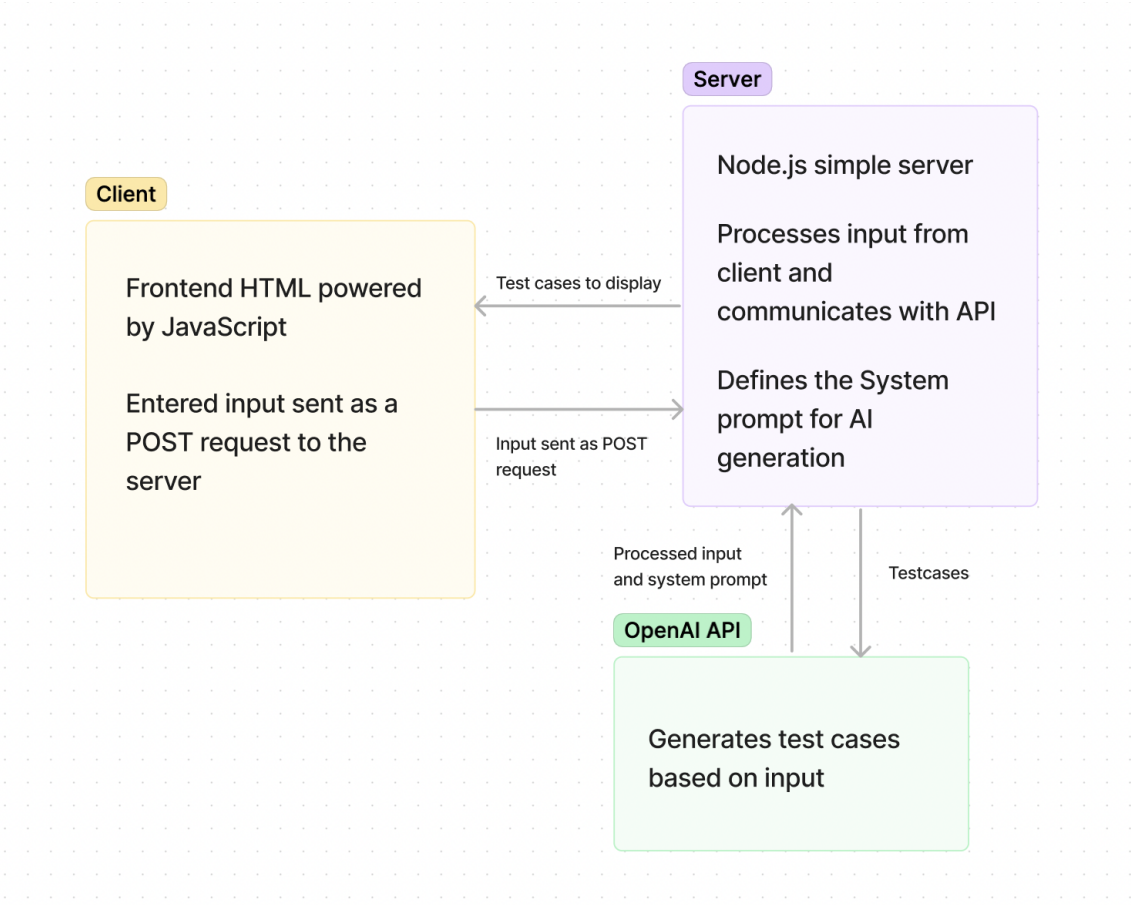

Figure 2 A diagram of the system structure, showing the client, server, and external API

3.2.4 Evaluation of the application

To evaluate the need for an application like this and the value of using AI for creating test cases, a few employees at Amaceit got access to the prototype to try it out and answer several questions. The questions were presented in a structured interview with questions related to the application prototype and the employees' way of working as consultants with test creation [10]. The questions that were asked in the interview were open-ended to not restrict anyone's thought process and allow for more detailed responses [19]. The interview questions together with the user testing allowed for a better understanding of the need for an AI service like the one created. It also helped show what kind of value AI can have in a consultancy business for software engineers.

4 Results and Discussion

In this section, the results of this thesis project are presented together with some thoughts to discuss the results. Section 4.1 presents the findings from the literature research, organized by research questions to provide targeted answers based on the review in section 3.1.1. Following that, section 4.2 presents the outcomes of the practical implementation, detailing the developed prototype, its use cases, and proper usage guidelines. Finally, the results of the application analysis are provided, including feedback from test users at Amaceit.

4.1 Theoretical results

The literature review highlights various AI techniques and AI tools utilized in software development, each offering distinct benefits and contributions across different stages of the process. The following results are divided by the earlier stated research questions in section 1.2, presented in organized lists for easy interpretation.

RQ1: In what stages of the development process is AI utilized today and in what way?

AI is utilized to some extent in all stages of the software development process. Different techniques are used for various purposes, presented in the following list.

Planning and Requirements Gathering:

- Generative AI can be utilized during planning to analyze historical data and estimate task durations based on previous patterns.
- Generative AI can also be utilized in requirements gathering to define project scopes efficiently, simplify proposal writing, and create prototypes.
- Natural language processing (NLP) can be used to specify requirements from written text.
- Machine learning can be used to analyze user behavior for requirements gathering.
- Search-based software engineering (SBSE) can be applied to optimize software requirements based on specific metrics in requirements gathering.

Design:

- AI can assist in creating system designs by providing insights for selecting architecture patterns and generating system designs from user-provided definitions.
- AI can also analyze design weaknesses, suggest improvements, assist in performance testing, and help create UX mockups.
- SBSE can assist in creating new system designs and optimizing designs based on various metrics.

Development:

- AI can be utilized to convert system design into code, provide suggestions for code snippets and algorithmic improvements, check for syntax and formatting errors, and assist in debugging.
- Generative AI tools and LLMs such as ChatGPT, Bard, Bing, and GitHub Copilot are extensively used to enhance code efficiency and productivity.
- Autoencoders can be utilized for processing data, reducing redundancy or unnecessary information, and simplifying complex data.

Testing and Quality Assurance:

- Generative AI can be utilized for generating and running tests on developed code to ensure requirements are met.
- AI can be used in regression- and user acceptance testing to identify faults before production.
- AI can help find edge cases during debugging and error handling of code and can execute automated test cases.
- Multiobjective search can be used to improve test case development and resolution by finding and prioritizing software crashes and suggesting fixes.
- Hierarchical clustering can be used to automatically fix common bugs based on past fix patterns.
- Machine learning can be utilized in automated software testing to streamline the process by executing test scripts repeatedly to gather and interpret the results.

- AI-powered program browsers can streamline testing and debugging by automatically proposing code changes.
- Fuzzing techniques can be utilized for systematic testing of programs by using invalid or random data.
- Deep neural networks can be employed to identify complex code defects and systematically improve code quality.
- SBSE can optimize test case development and resolution, ensuring thorough testing and higher software quality.

Deployment and Maintenance:

- Generative AI can aid in preparing for production release by organizing the components and breaking down the release into individual tasks.
- Generative AI can also help in monitoring performance during and after deployment and identifying areas for improvement post-deployment.
- AI tools can assist in finding and fixing production errors efficiently, ensuring system security.
- AI can also help to identify patterns by analyzing user behavior, ensuring developers can address issues early on, and offer insights into system performance and user engagement.
- SBSE optimizes maintenance processes, ensuring quick deployment of patches and proactive issue resolution.

RQ2: How can AI be used to streamline software development and software testing, specifically reducing time and cost?

AI offers many possibilities to aid in streamlining development processes which will be presented further in the following lists. In these results, the 'software development' includes the planning and designing phase as well as the actual coding process. The 'software testing' encompasses test cases and debugging, as well as maintenance and quality assurance.

Streamlining software development:

- AI techniques like generative AI and NLP can streamline project planning by analyzing historical data, estimating task durations, and defining project scopes efficiently.
- SBSE can optimize requirements gathering, leading to better project planning and cost management.
- AI-driven tools (e.g., GitHub Copilot, ChatGPT) can expedite system designs, coding processes, and syntax checking, ultimately reducing development time.
- AI can suggest code snippets and algorithmic improvements, improving efficiency and reducing costs.

Streamlining software testing:

- Generative AI can assist in automating test generation, regression testing, and user acceptance testing, thereby improving the efficiency of testing processes.
- Machine learning techniques can enhance test case development and resolution, improving software quality and reducing testing time.
- AI tools can aid in preparing for production release, monitoring performance, and identifying areas for improvement post-deployment, leading to efficient deployment processes.
- AI can assist in finding and fixing production errors efficiently, ensuring system security, and analyzing user behavior, thereby reducing maintenance costs and enhancing software reliability.

RQ3: What AI tools are commonly used in software development today and what are their benefits and challenges?

Today, there is a wide range of AI tools available, designed for software development and various other applications. This study focuses on eight of the most popular tools, detailed in section 3.1.1. The findings are presented through tables and graphs, showcasing the distinct properties of the different AI tools that were explored. Since the different studies used various techniques and parameters to measure the quality of the tools, they have been separated to compare the same parameters. Note that code correctness in Figure 4 and code effectiveness in Figure 6 are not measured in the same way. Refer to section 3.1.1 for an explanation of the percentages.

Comparison of Generative AI Tools

In Figures 3 and 4, the distinct properties and differences between the explored code generation tools are displayed. Figure 3 indicates that Amazon CodeWhisperer was the quickest to fix code smells, taking 5.6 minutes, while GitHub Copilot was the slowest at 9.1 minutes. The figure also shows that all tools had a similar decrease in correctness when lower-quality parameters were used but ChatGPT showed the least performance decrease when using generic function names. In Figure 4, it's evident that ChatGPT also achieved the highest correctness score at 65.2%, while Amazon CodeWhisperer achieved the lowest score at 31.1%.

Code Generation tools	*Time to fix code smells (min)*	*Lower-quality parameters*	*Generic/placeholder function names*
GitHub Copilot	9.1	Decreased correctness	Decreased performance
Amazon CodeWhisperer	5.6	Decreased correctness	Decreased performance
ChatGPT	8.9	Decreased correctness	Slightly decreased performance

Figure 3 Table summarizing various code generation tools based on their time to fix code smells and correctness and performance impacts due to different parameters and function names.

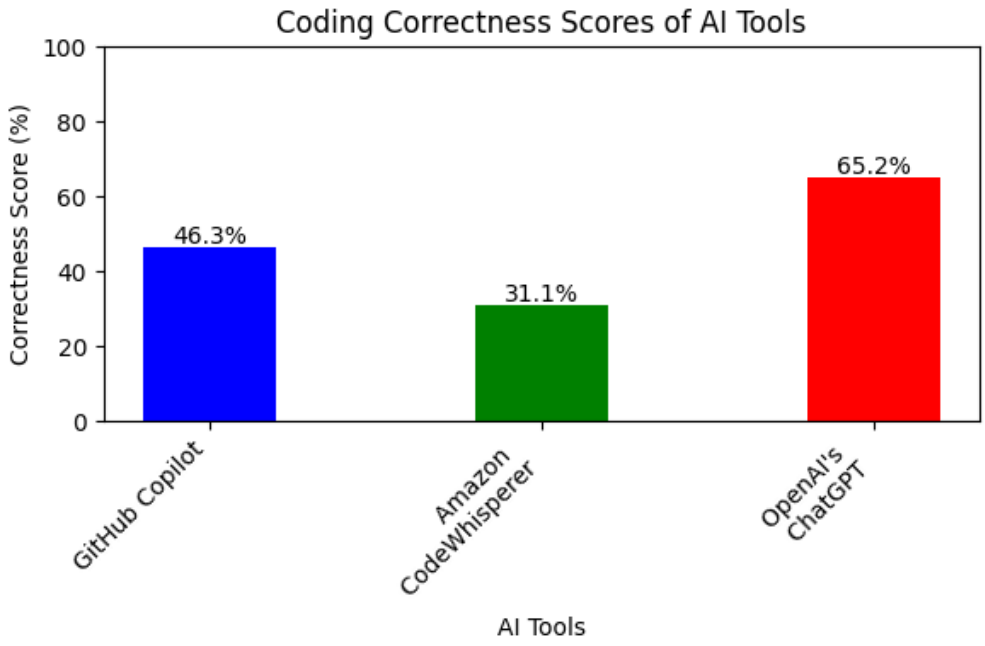

Figure 4 Plot showing the correctness scores of the following code generation tools: Github Copilot, Amazon CodeWhisperer, and ChatGPT.

For the comparison of different large language model tools, various aspects are illustrated in Figures 5 and 6. A trend in Figure 5 shows that all GPT-based models excel at producing short and concise code, unlike their competitors Google's Bard and Anthropic's Claude, which tend to generate unnecessarily long and complicated code. Among all models, Microsoft's Bing produced the lowest average number of lines of code (LOC), indicating higher quality code. Additionally, ChatGPT based on the GPT-4 model was the only one to successfully resolve errors in the code after receiving user feedback. In contrast, Bard had the highest average number of LOC and only acknowledged errors without correcting them, as shown in Figure 5. Figure 6 displays the effectiveness scores of all LLM tools, where the GPT-4 model of ChatGPT scored the highest at 87.51%, closely followed by ChatGPT-3 and Bing (also based on GPT-4). Claude exhibited the lowest score at 71.43% with Bard at second to last place.

LLM tools	*Code length*	*Benefits*	*Challenges*
ChatGPT (GPT-3.5)	Short and concise		
ChatGPT (GPT-4)	Short and concise	Improves and resolves all errors after user feedback	
Google's Bard	Longer and unnecessarily complicated		Highest avg. no. of LOC, indicating lower quality Acknowledges errors but does not correct them
Microsoft's Bing (GPT-4)	Short and concise	Lowest avg. no. of LOC, indicating higher quality	
Anthropic's Claude	Longer and unnecessarily complicated		

Figure 5 Table summarizing various LLM tools based on the generated code length and various benefits and challenges of the tools.

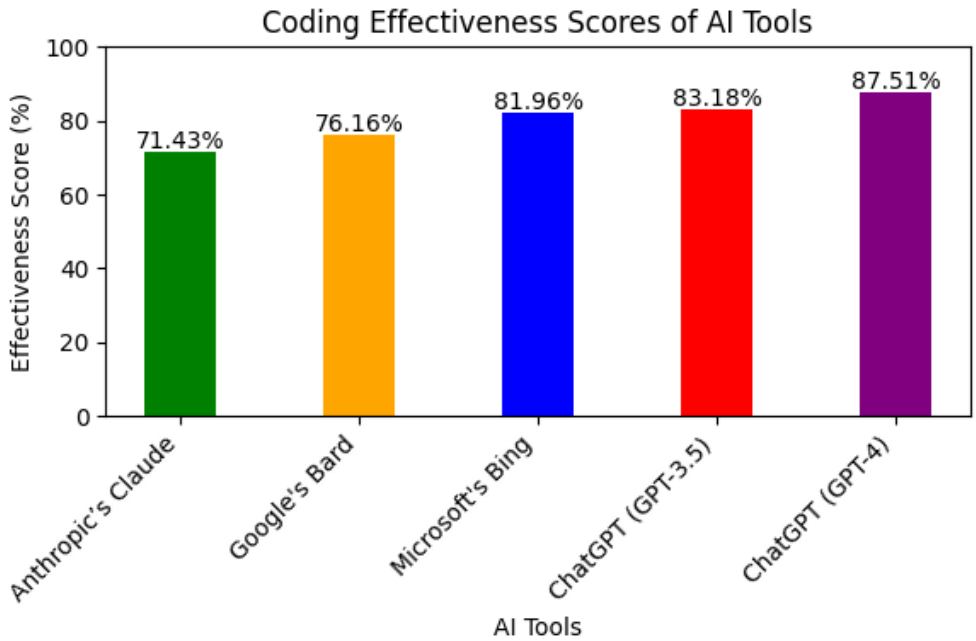

Figure 6 Plot showing the effectiveness scores of the following code generation tools: Claude, Bard, Bing, ChatGPT (3.5) and ChatGPT (4).

In the analysis of the code generation tools, Figures 4 and 6 highlight the differences in performance among the tools. While ChatGPT and GPT-based models generally show superior results compared to others, it is important to acknowledge that interpreting these performance scores can be somewhat subjective. The relative effectiveness of these tools is clear, but determining what constitutes a "good" score in absolute terms is challenging. For instance, the highest correctness scores of 65.2% and 87.51% might seem modest compared to benchmarks for human programmers. This brings to light the difficulty in setting standard performance thresholds for AI tools, as there is no clear consensus on what level of performance should be considered acceptable.

4.2 Practical results

The practical implementation resulted in a proof of concept (PoC) version of a web page built in JavaScript, utilizing the OpenAI API to generate test cases as shown in Figure 7. For the input, the user starts by writing in free text the preferred programming language and a suitable test library to use. Next, the user can either upload a file containing code or write free text in the text box. Once the input is complete, pressing the "Generate test cases" button produces test cases in the correct format, which are dis-

played in the text box at the bottom. All steps of using the application are shown in Figure 8.

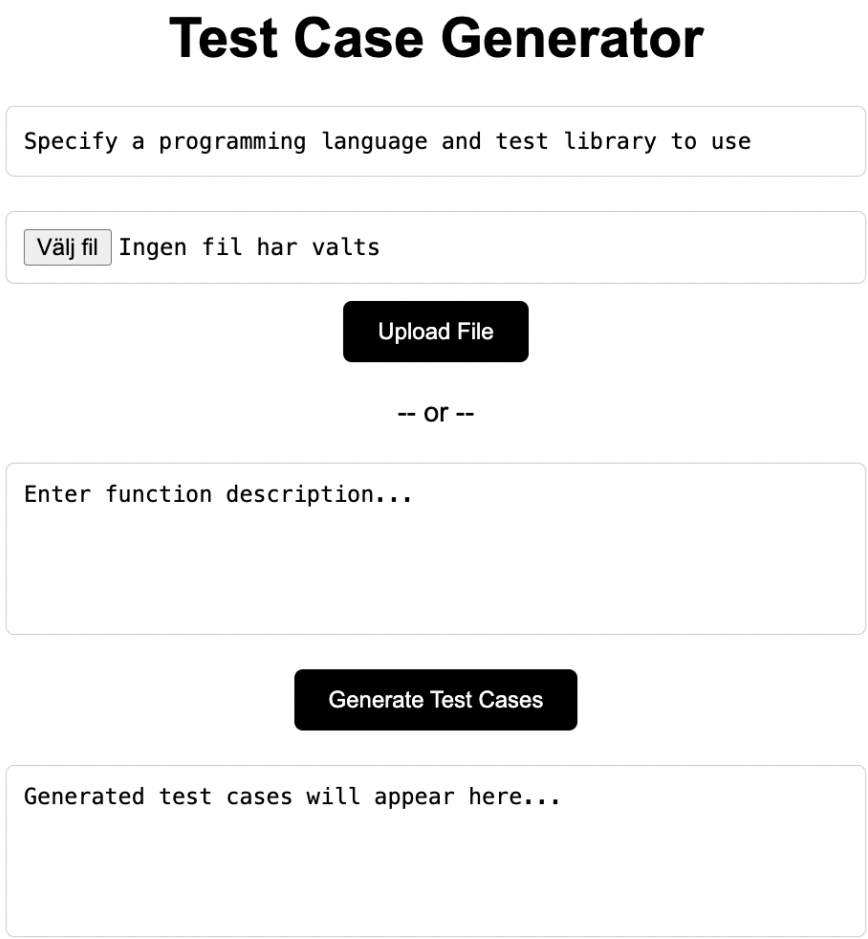

Figure 7 Default view of the Test Case Generator.

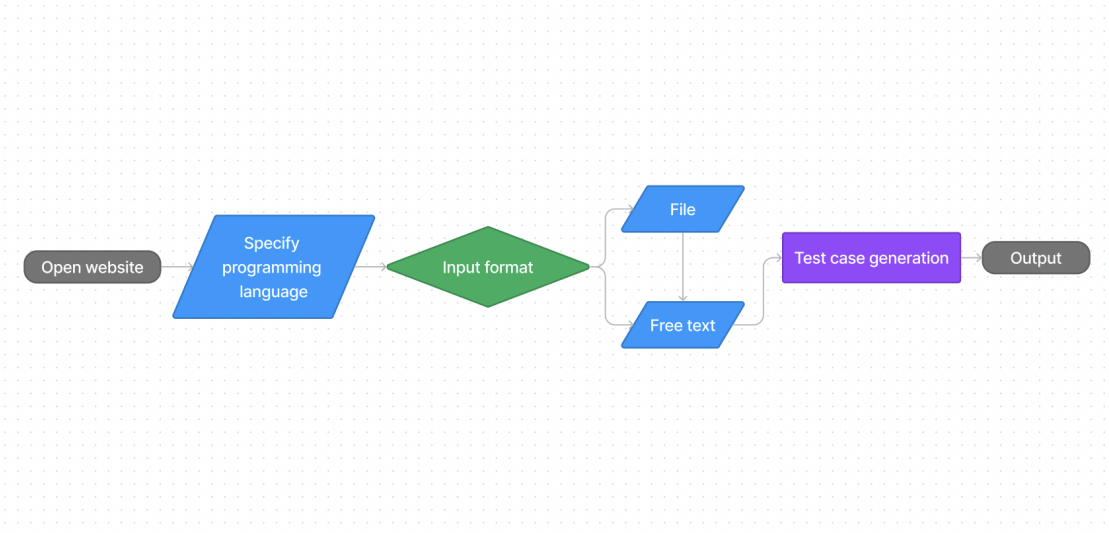

Figure 8 A diagram showing the flow of the application's use. The grey boxes indicate a start or end, the blue an input, the green a choice to make, and the purple a process happening.

In Figure 9, the results from using the service with a JavaScript example are shown. A JavaScript file has been uploaded and its contents have been printed in the first text box. Then the button has been pressed and test cases written using the JEST library have been printed. The user can at this stage easily copy the test cases from the second text box and paste them into any code file or workspace to utilize them in their work.

Test Case Generator

```
javascript JEST
```

Välj fil example.js

Upload File

-- or --

```
/**
* Computes the factorial of a non-negative integer.
*
* @param {number} n - The non-negative integer for which factorial is to be calculated.
* @returns {number} The factorial of n.
*/
function calculateFactorial(n) {
if (n < 0) {
throw new Error("Cannot calculate factorial of a negative number");
}
if (n === 0 || n === 1) {
return 1;
}
let factorial = 1;
for (let i = 2; i <= n; i++) {
factorial *= i;
}
return factorial;
}
```

Generate Test Cases

```
const calculateFactorial = require('./calculateFactorial');

describe('calculateFactorial function', () => {
test('should throw an error if the input is negative', () => {
expect(() => calculateFactorial(-1)).toThrow('Cannot calculate factorial of a negative number');
});

test('should return 1 if the input is 0', () => {
expect(calculateFactorial(0)).toBe(1);
});

test('should return 1 if the input is 1', () => {
expect(calculateFactorial(1)).toBe(1);
});

test('should return the correct factorial for numbers greater than 1', () => {
expect(calculateFactorial(2)).toBe(2);
expect(calculateFactorial(3)).toBe(6);
expect(calculateFactorial(4)).toBe(24);
expect(calculateFactorial(5)).toBe(120);
});
});
```

Figure 9 An example of the Test Case Generator, using a JavaScript file and generating test cases in JEST.

Evaluation of prototype

To evaluate the usability of the implemented prototype and the need for the service, an interview was conducted together with user testing by employees at Amaceit as explained in section 3.2.4. In total, five people participated in the interview. A summary of what the participants answered is presented in the following section, all 11 questions that were asked can be found in Appendix A.1.

The participants, all working as full-stack developers or programmers, answered that they usually spend between 5 to 15% of their working hours writing tests. Their current processes for writing tests vary: some usually work in .Net using libraries such as MSTest and xUnit, while others are working in Angular using test libraries called Karma or Jasmine. One participant specifically mentioned working with backend C# and using Visual Studio Unit Test Project for test cases. Additionally, another participant highlighted using the triple-A method (Arrange, Act, Assert) for their testing, believing it to be a straightforward and easy method to use.

A theme among all participants was that they frequently skip writing test cases due to time constraints or other priorities. One participant noted that this occurs very often, with tests typically not being written for new features unless they are complex enough to justify the additional effort. Similarly, another participant mentioned that they regularly skip writing tests, opting for manual testing instead, except when it is mandatory for a course (referring to when said participant was recently in school) or specifically required by their employer.

When asked about the potential impact of integrating an AI tool into their workflow, participants generally agreed that AI could improve efficiency by speeding up test writing and increasing test coverage, potentially leading to cost reductions. However, it was noted that developers would still need to verify the accuracy of the tests. This highlights how AI currently serves as a valuable assistant to human users, specifically in coding, but is not yet in a state to be used fully on its own. One participant mentioned that in their current job, the AI tool would not affect cost but would improve the efficiency and quality of tests by making test case writing less time-consuming. Another participant highlighted that the tool could be particularly beneficial if it was compatible with specific backend tests, such as those written in C#, pointing out the specific language this participant mostly works in. These insights underscore how AI tools can fill gaps where tests are currently omitted, as noted by participants earlier. By automating these processes, developers' workload could be alleviated.

Regarding the quality of test cases, participants generally believed that the AI tool could enhance quality. One participant noted that the quality would only improve since no test cases are currently being written, while another mentioned that the tool would save time

in creating test cases, contributing to better quality overall. A common response among the participants was that it is difficult to assess the quality of the generated tests at this stage since the application is only a PoC; however, most believed that it works well for simpler functions at this time.

One important question that was asked explored the potential challenges or barriers that could be associated with using this kind of AI tool. A notable concern highlighted by several participants to adopt this tool in a consultancy context includes concerns about companies being unwilling to share code with OpenAI and the need for contractor permission due to confidential data. The issue of privacy regarding AI is to be expected. Many companies hesitate to share code with AI providers due to trust and security concerns, fearing exposure or misuse of sensitive information. They want to maintain control over their data and keep it within secure environments. Anonymizing data before AI processing and offering on-premise versions of AI tools could provide a solution and help alleviate these privacy concerns. However, there are no significant barriers anticipated for in-house projects since these would not have the same privacy concern, according to some participants.

When asked for any additional feedback or comments, most participants did not have anything to add at the moment. However, one participant suggested that having this type of AI functionality as an intuitive plugin for IDEs would increase the amount of people interested in using it. This idea was mentioned by the same participant when asked if they would be interested in using the tool in future projects. The concept was then further explained as a functionality, preferably in Visual Studio Code, where one could press on a function or class and it automatically would generate test cases. This idea was particularly interesting since it was one of the initial plans for the practical implementation but unfortunately was not possible for this specific project.

5 Conclusions

The results from this thesis project have demonstrated significant advancements in the application of AI techniques across various stages of the software development lifecycle. The literature review revealed that AI is increasingly integrated into planning, design, development, testing, and maintenance processes, enhancing efficiency and effectiveness. For instance, AI techniques like generative AI, NLP, and SBSE optimize project planning and requirements gathering, while neural networks and LLMs improve design and coding processes by suggesting code snippets and identifying potential improvements. AI as a tool is regarded as a valuable asset in software engineering, yet it still needs human supervision to ensure accuracy and reliability. They exhibit the potential to boost efficiency and minimize time spent on straightforward tasks, thereby serving as effective assistants to human programmers. These findings underscore the dual aspects of AI in software development: a tool that enhances productivity while still requiring human intervention to ensure high-quality outcomes.

Additionally, AI-driven tools have been shown to expedite coding and debugging, although their performance varies based on the quality of input and specific use cases. These tools not only streamline development tasks but also significantly reduce the time and cost associated with software creation and maintenance. Among the popular AI tools, GPT models, especially ChatGPT (GPT-4), demonstrated superior performance in code correctness, efficiency in fixing code smells, and generating concise code. In contrast, Amazon CodeWhisperer, Bard, and Claude showed weaker performance in terms of correctness and code quality.

However, interpreting these performance scores can be challenging, as benchmarks for AI tools are not well-defined. The highest scores achieved highlight the relative strengths of these models but may still fall short compared to human programming standards. This underscores the need for ongoing evaluation to better understand what constitutes an acceptable performance level for AI in software development.

The practical implementation of a webpage utilizing OpenAI API proved successful as a proof of concept. It demonstrated the feasibility of using AI for generating test cases. The generated test cases were found to be accurate and compatible with the specified libraries, showcasing the prototype's practical usefulness. However, further development is necessary to optimize its functionality and see its full potential for professional environments. The prototype showed promise in improving efficiency by ensuring that test cases are generated promptly, although there remains a need to enhance the quality assurance aspect of generated tests.

One of the notable challenges identified relates to data privacy and confidentiality. Con-

cerns were raised about sharing sensitive information with external AI service providers like OpenAI and addressing these concerns will be important for a broader adoption.

For future work, it would be valuable to develop the AI service further to support more complex functions and larger datasets. The current AI service, designed as a proof of concept, demonstrated its potential on a smaller scale, but testing on a broader scale would yield a more accurate assessment of its effectiveness in a consultancy business. Additionally, integrating AI into IDEs as a plugin holds significant promise. This integration could streamline developers' workflows by enabling them to generate test cases directly within their development environments, such as Visual Studio Code. Expanding research to include a wider range of companies and consultants beyond the current small sample could provide deeper insights into real-world applications. This approach would capture a broader array of opinions and experiences, leading to a more comprehensive understanding of AI's impact across different consultancy settings and refining the AI service's utility and effectiveness.

Additionally, exploring specialized AI tools for software development could offer valuable insights. Similar to Copilot, there are other popular AI tools designed to assist developers by providing code suggestions within the IDE or workspace. Unlike LLMs that generate entire codebases, these tools focus on making coding tasks easier by offering context-aware suggestions. Investigating these tools could reveal their impact on developer productivity and workflow efficiency.

Another important area for future research is the issue of copyright related to AI-generated code. Many AI tools are trained on data sourced from the internet, leading to debates about the originality and ownership of the generated code. Understanding how AI-generated content intersects with copyright laws and intellectual property rights is crucial for addressing potential legal and ethical concerns.

Lastly, expanding the scope of literature research would be beneficial due to the vast and continually evolving nature of this field. Examining case studies or conducting new ones could provide valuable insights into real-world applications. This approach would be particularly enlightening when comparing various AI tools and generative AI platforms over an extended period, capturing their ongoing advancements and refinements.

6 References

[1] I. Ahmed, S. Ghorashi, and C. Jensen, "An exploration of code quality in foss projects," in *Open Source Software: Mobile Open Source Technologies*, 2014, pp. 181–190.

[2] Amazon Web Services (AWS), "Amazon Web Services (AWS): What is GPT?" https://aws.amazon.com/what-is/gpt/, 2023.

[3] Amazon Web Services (AWS), "What is SDLC?" https://aws.amazon.com/what-is/sdlc/, 2023.

[4] H. H. Ammar, W. Abdelmoez, and M. S. Hamdi, "Software Engineering Using Artificial Intelligence Techniques: Current State and Open Problems," in *Proceedings of the First Taibah University International Conference on Computing and Information Technology*, 2012, p. 52.

[5] Atlassian, "Waterfall Methodology: A Comprehensive Guide," https://www.atlassian.com/agile/project-management/waterfall-methodology.

[6] M. Barenkamp, J. Rebstadt, and O. Thomas, "Applications of AI in Classical Software Engineering," *AI Perspectives*, vol. 2, 2020.

[7] cdteliot, "AI in Entertainment: Automated Content Generation and Personalized Recommendations," https://www.perplexity.ai/page/ai-in-entertainment-personaliz-ltsxojxRQpOgW1Fl9XaIsA, 2024.

[8] C. E. A. Coello, M. N. Alimam, and R. Kouatly, "Effectiveness of ChatGPT in Coding: A Comparative Analysis of Popular Large Language Models," in *Digital*, vol. 4, no. 1, 2024, pp. 114–125.

[9] S. Feuerriegel, J. Hartmann, C. Janiesch *et al.*, "Generative ai," in *Bus Inf Syst Eng*, vol. 66, no. 1, 2024, pp. 111–126.

[10] T. George and J. Merkus, "Structured Interview | Definition, Guide & Examples," https://www.scribbr.com/methodology/structured-interview/, 2022.

[11] Ghent University, "Searching: what is the snowball search method?" https://onderzoektips.ugent.be/en/tips/00002030/, 2023.

[12] M. Harman, "The Role of Artificial Intelligence in Software Engineering," in *2012 First International Workshop on Realizing AI Synergies in Software Engineering (RAISE)*, 2012, pp. 1–6.

[13] IBM, "What is artificial intelligence (AI)?" https://www.ibm.com/topics/artificial-intelligence.

[14] IBM, "What is machine learning? (ML)?" https://www.ibm.com/topics/machine-learning.

[15] M. Koroteev, "Bert: A review of applications in natural language processing and understanding," 2021.

[16] M. A. Kuhail, S. S. Mathew, A. Khalil, J. Berengueres, and S. J. H. Shah, ""Will I Be Replaced?" Assessing ChatGPT's Effect on Software Development and Programmer Perceptions of AI Tools," *Science of Computer Programming*, 2024.

[17] C. Manning, "Artificial Intelligence Definitions," https://hai.stanford.edu/sites/default/files/2020-09/AI-Definitions-HAI.pdf, 2020.

[18] I. Ozkaya, "The Next Frontier in Software Development: AI-Augmented Software Development Processes," in *IEEE Software*, vol. 40, no. 4, 2023, pp. 4–9.

[19] M. Rosala, "Open-Ended vs. Closed Questions in User Research," https://www.nngroup.com/articles/open-ended-questions/, 2024.

[20] V. Shah, "Towards Efficient Software Engineering in the Era of AI and ML: Best Practices and Challenges," in *Science of Computer Programming*, vol. 3, no. 3, 2019, pp. 63–78.

[21] A. Shastri Pothukuchi, L. V. Kota, and V. Mallikarjunaradhya, "Impact of Generative AI on the Software Development Lifecycle (SDLC)," in *International Journal of Creative Research Thoughts*, vol. 11, no. 8, 2023, p. 5.

[22] B. Sundarambal, C. Karthikeyini, R. Bommi, S. Subramanian, and M. Lokesh, "AI Based Content Management System," in *2023 International Conference on Data Science, Agents Artificial Intelligence (ICDSAAI)*, 2023, pp. 1–8.

[23] C. Tantithamthavorn, J. Jiarpakdee, and J. Grundy, "Explainable AI for Software Engineering," 2020.

[24] P. Tseluyko, "How to Decide on AI API: ChatGPT, Gemini, or Claude," https://merge.rocks/blog/how-to-decide-on-ai-api-chatgpt-gemini-or-claude, 2024.

[25] D. M. West and J. R. Allen, "How Artificial Intelligence Is Transforming the World," https://www.brookings.edu/articles/how-artificial-intelligence-is-transforming-the-world/#_edn2, 2018.

[26] B. Yetiştiren, I. Özsoy, M. Ayerdem, and E. Tüzün, "Evaluating the Code Quality of AI-Assisted Code Generation Tools: An Empirical Study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT," 2023.

[27] W. X. Zhao, K. Zhou, J. Li *et al.*, "A Survey of Large Language Models," 2023.

A Appendix

A.1 Interview questions

1. What is your role at the company, and how often do you engage in writing or reviewing code?
2. On average, how much of your time is spent writing test cases during a typical project? (e.g., percentage of time, hours per week)
3. Can you describe your current process for writing test cases? What tools or frameworks do you use?
4. Do you ever find yourself skipping writing test cases due to time constraints or other priorities? If so, how often does this occur?
5. How do you think integrating this kind of AI tool into your workflow could impact your current processes in terms of efficiency, cost, and quality? Please explain your reasoning.
6. Do you believe that this kind of tool could affect the quality (better or worse) of the test cases written for your projects? Please explain your reasoning.
7. What are your initial impressions of the test case generation application? Is it intuitive to use, and are the generated test cases relevant and useful?
8. How does the quality of the AI-generated test cases compare to those you would write manually? Any noticeable differences in thoroughness or accuracy?
9. Would you consider using this kind of AI tool in your future projects? What features or improvements would make you more likely to adopt it?
10. What barriers, if any, do you foresee in adopting this kind of tool in your work, specifically from a consultancy perspective?
11. Do you have any additional comments or suggestions regarding the test case generation application or the concept of using AI for test case creation?